

Implementación de funciones

Sumador cuántico

Naomi Itzel Reyes Granados

Facultad de Ingeniería

Dada una función $f : X \rightarrow Y$, con X finito y suponiendo que conocemos $f(x)$ para cada elemento $x \in X$.

¿Cómo podrías definir la función f de manera clásica en una computadora?

Dado que el dominio es finito y conocemos el valor de $f(x)$ para cada $x \in X$ podemos usar una cadena de condiciones del tipo:

`if  $x = x_j$  then return  $f(x_j)$ .`

Dado que el dominio es finito y conocemos el valor de $f(x)$ para cada $x \in X$ podemos usar una cadena de condiciones del tipo:

`if  $x = x_j$  then return  $f(x_j)$ .`

Pregunta

En computación cuántica, ¿Qué compuerta abstrae la idea de una condicional?

Dado que el dominio es finito y conocemos el valor de $f(x)$ para cada $x \in X$ podemos usar una cadena de condiciones del tipo:

`if  $x = x_j$  then return  $f(x_j)$ .`

Pregunta

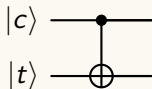
En computación cuántica, ¿Qué compuerta abstrae la idea de una condicional?

La puertas controladas: *CNOT*

La semántica de la compuerta $CNOT(c, t) = CNOT |c\rangle |t\rangle$ es:

if $c = 1$ then return $\neg t$.

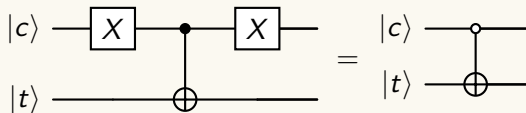
Cuya notación en circuito es:



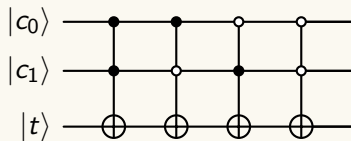
Existe la compuerta que en lugar de verificar que el estado del cúbit de control sea 1 verifica que sea 0: $\hat{CNOT}(c, t) = (\sigma_x \otimes I)CNOT(\sigma_x \otimes I)|c\rangle|t\rangle$ semánticamente:

`if  $c = 0$  then return  $\neg t$ .`

Cuya notación en circuito es:

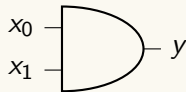


Estas puertas cuánticas se pueden generalizar para que verifiquen más de un cúbit (aumentar el número de cúbits control) y mezclarlos:



Implementación AND

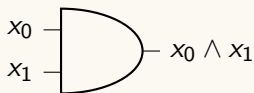
Una de las puertas lógicas clásicas más famosas es el AND, sin embargo no se ha planteado como una puerta cuántica,



¿Porqué?

Implementación AND

Una de las puertas lógicas clásicas más famosas es el AND, sin embargo no se ha planteado como una puerta cuántica,



¿Porqué? No es reversible!

Operadores cuánticos

Si U es un operador cuántico entonces:

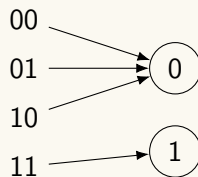
$$UU^\dagger = U^\dagger U = I.$$

Por tanto, $U^{-1} = U^\dagger$.

Implementación AND

Analicemos la tabla de verdad del AND:

x_0	x_1	$x_0 \wedge x_1$
0	0	0
0	1	0
1	0	0
1	1	1



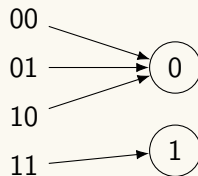
Al observar 0, no sabemos si la entrada fue 00, 01 o 10.

¿Cómo podríamos modificar la puerta clásica para no perder información?

Implementación AND

Analicemos la tabla de verdad del AND:

x_0	x_1	$x_0 \wedge x_1$
0	0	0
0	1	0
1	0	0
1	1	1



Al observar 0, no sabemos si la entrada fue 00, 01 o 10.

¿Cómo podríamos modificar la puerta clásica para no perder información?

Tomando la entrada como parte de la salida.

Implementación AND

Analicemos la tabla de verdad del AND:

x_0	x_1	$x_0 \wedge x_1$
0	0	0
0	1	0
1	0	0
1	1	1

Si queremos construir una compuerta cuantica que corresponda al *AND*,

¿Cuántos cúbits serán necesarios?

Implementación AND

Analizamos la tabla de verdad del AND:

x_0	x_1	$x_0 \wedge x_1$
0	0	0
0	1	0
1	0	0
1	1	1

Si queremos construir una compuerta cuantica que corresponda al *AND*,

¿Cuántos cúbits serán necesarios? 3

¿Cómo la implementarías?

Implementación AND

Analicemos la tabla de verdad del AND:

x_0	x_1	$x_0 \wedge x_1$
0	0	0
0	1	0
1	0	0
1	1	1

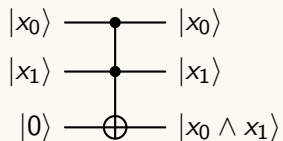
Si queremos construir una compuerta cuantica que corresponda al *AND*,

¿Cuántos cúbits serán necesarios? 3

¿Cómo la implementarías? Usando las puertas controladas. Si asumimos que el cúbit donde se guardará la información se inicializa en el estado $|0\rangle$, basta con "cachar" los estados de las entradas que modifican su valor al estado $|1\rangle$.

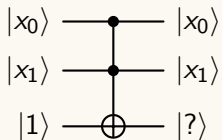
Implementación AND

Por lo que el circuito generado se vería de la siguiente manera:



Implementación AND

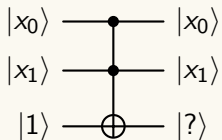
Por lo que el circuito generado se vería de la siguiente manera:



¿Qué pasaría si el último cúbit se inicializa en el estado $|1\rangle$ en lugar de $|0\rangle$?

Implementación AND

Por lo que el circuito generado se vería de la siguiente manera:

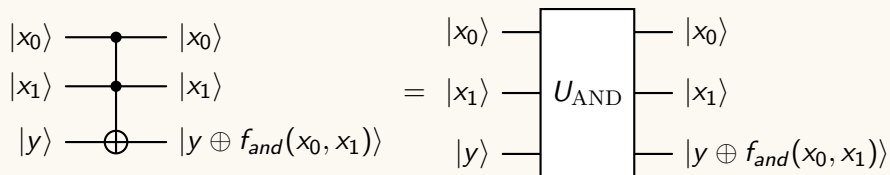


¿Qué pasaría si el último cúbit se inicializa en el estado $|1\rangle$ en lugar de $|0\rangle$?

Se intercambia el valor esperado de la función.

Implementación AND

Generalizando el valor con el que se inicia el cúbit donde se guarda el valor del resultado tenemos:



El operador $\oplus$ representa la suma modulo 2.

A los cúbits que se ocupan para guardar el resultado de la función les llamamos cúbits de ancilla. En este caso el cúbit $|y\rangle$ sería nuestro cúbit de ancilla.

Haciendo una implementación análoga, ¿Cómo implementarías un circuito que corresponda a la suma de dos cúbits?

- ¿Cuántos cúbits correspondientes a las entradas necesitarías?
- ¿Cuántos cúbits correspondientes a la salida (cúbits de ancilla) necesitarías?

Haciendo una implementación análoga, ¿Cómo implementarías un circuito que corresponda a la suma de dos cúbits?

- ¿Cuántos cúbits correspondientes a las entradas necesitarías? 2
- ¿Cuántos cúbits correspondientes a la salida (cúbits de ancilla) necesitarías? 2

Ya identificada esta información se construirá la tabla de verdad correspondiente a la función f_{SUM} .

Implementación Suma

Construcción de la tabla de verdad de $f_{SUM}(x_0x_1) = y_0y_1$:

x_0	x_1	y_0	y_1
0	0		
0	1		
1	0		
1	1		

Implementación Suma

Construcción de la tabla de verdad de $f_{SUM}(x_0x_1) = y_0y_1$:

x_0	x_1	y_0	y_1
0	0	0	0
0	1		
1	0		
1	1		

Implementación Suma

Construcción de la tabla de verdad de $f_{SUM}(x_0x_1) = y_0y_1$:

x_0	x_1	y_0	y_1
0	0	0	0
0	1	0	1
1	0		
1	1		

Implementación Suma

Construcción de la tabla de verdad de $f_{SUM}(x_0x_1) = y_0y_1$:

x_0	x_1	y_0	y_1
0	0	0	0
0	1	0	1
1	0	0	1
1	1		

Implementación Suma

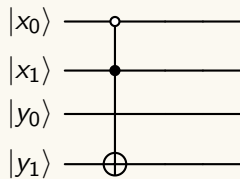
Construcción de la tabla de verdad de $f_{SUM}(x_0x_1) = y_0y_1$:

x_0	x_1	y_0	y_1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Con esta tabla pasamos a cosntruir el circuito correspondiente haciendo uso de los operadores controlados.

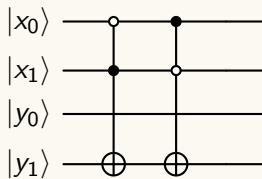
Implementación Suma

Construcción del circuito sumador:



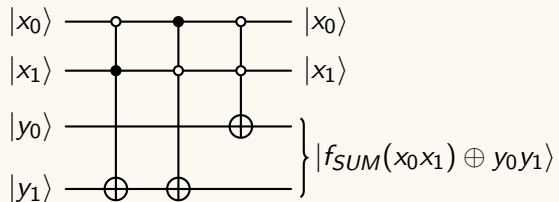
Implementación Suma

Construcción del circuito sumador:



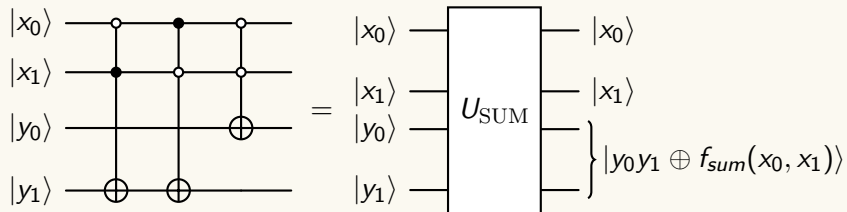
Implementación Suma

Construcción del circuito sumador:



Implementación Suma

Construcción del circuito sumador:



- 1 Podemos generalizar este proceso para sumas de números más grandes.

Primeras conclusiones

- 1 Podemos generalizar este proceso para sumas de números más grandes.
- 2 Más aún podemos generalizar para cualquier función clásica f . Por lo que podemos concluir que en computación cuántica podemos hacer al menos lo mismo que en computación clásica.

Hasta este punto solo hemos trabajado con estados clásicos, ¿Dónde se podría asomar alguna "ventaja" cuántica?

Hasta este punto solo hemos trabajado con estados clásicos, ¿Dónde se podría asomar alguna "ventaja" cuántica?

Evaluar en estados en superposición.

Preparamos un estado en superposición de todas las entradas con la misma amplitud:

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|00\rangle.$$

¿Cuál será el estado después de aplicar U_{sum} ?

Preparamos un estado en superposición de todas las entradas con la misma amplitud:

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|00\rangle.$$

Aplicando el operador:

$$U_{sum} \left(\frac{|00\rangle |00\rangle + |10\rangle |00\rangle + |10\rangle |00\rangle + |11\rangle |00\rangle}{\sqrt{2}} \right) = \frac{U_{sum}(|00\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|11\rangle |00\rangle)}{\sqrt{2}} \quad (1)$$

Preparamos un estado en superposición de todas las entradas con la misma amplitud:

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|00\rangle.$$

Aplicando el operador:

$$U_{sum} \left(\frac{|00\rangle |00\rangle + |10\rangle |00\rangle + |10\rangle |00\rangle + |11\rangle |00\rangle}{\sqrt{2}} \right) = \frac{U_{sum}(|00\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|11\rangle |00\rangle)}{\sqrt{2}} \quad (2)$$

$$= \frac{|00\rangle |00\rangle + |10\rangle |01\rangle + |10\rangle |01\rangle + |11\rangle |10\rangle}{\sqrt{2}} \quad (3)$$

Preparamos un estado en superposición de todas las entradas con la misma amplitud:

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|00\rangle.$$

Aplicando el operador:

$$U_{sum} \left(\frac{|00\rangle |00\rangle + |10\rangle |00\rangle + |10\rangle |00\rangle + |11\rangle |00\rangle}{\sqrt{2}} \right) = \frac{U_{sum}(|00\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|11\rangle |00\rangle)}{\sqrt{2}} \quad (4)$$

$$= \frac{|00\rangle |00\rangle + |10\rangle |01\rangle + |10\rangle |01\rangle + |11\rangle |10\rangle}{\sqrt{2}} \quad (5)$$

¡Obtenemos una superposición de resultados!

Preparamos un estado en superposición de todas las entradas con la misma amplitud:

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|00\rangle.$$

Aplicando el operador:

$$U_{sum} \left(\frac{|00\rangle |00\rangle + |10\rangle |00\rangle + |10\rangle |00\rangle + |11\rangle |00\rangle}{\sqrt{2}} \right) = \frac{U_{sum}(|00\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|11\rangle |00\rangle)}{\sqrt{2}} \quad (6)$$

$$= \frac{|00\rangle |00\rangle + |10\rangle |01\rangle + |10\rangle |01\rangle + |11\rangle |10\rangle}{\sqrt{2}} \quad (7)$$

¿Qué pasa cuando midamos?

Preparamos un estado en superposición de todas las entradas con la misma amplitud:

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|00\rangle.$$

Aplicando el operador:

$$U_{sum} \left(\frac{|00\rangle |00\rangle + |10\rangle |00\rangle + |10\rangle |00\rangle + |11\rangle |00\rangle}{\sqrt{2}} \right) = \frac{U_{sum}(|00\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|10\rangle |00\rangle) + U_{sum}(|11\rangle |00\rangle)}{\sqrt{2}} \quad (8)$$

$$= \frac{|00\rangle |00\rangle + |10\rangle |01\rangle + |10\rangle |01\rangle + |11\rangle |10\rangle}{\sqrt{2}} \quad (9)$$

¿Qué pasa cuando midamos? Se colapsa a un solo resultado, por lo que pareciera que no ganamos. Sin embargo veremos que con el uso de la interferencia cuántica ¡podremos sacar "ventaja"!

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1		
1	1	0		
1	1	1		

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0		
1	1	1		

Sumador completo

Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1		

Sumador completo

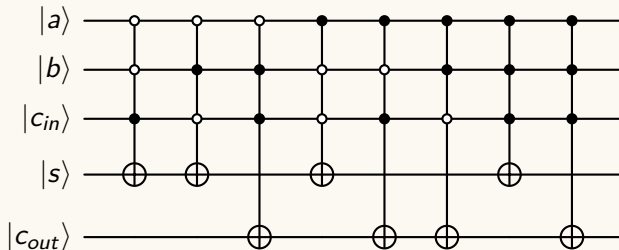
Como último ejemplo veremos como implementar un sumador con acarreo. Sumamos tres cúbits: a , b y el acarreo de entrada c_{in} .

$$s = a \oplus b \oplus c_{\text{in}}, \quad c_{\text{out}} = ab \oplus c_{\text{in}}(a \oplus b).$$

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sumador completo

Usamos auxiliares $s = c = 0$ y preservamos las tres entradas.



¡Gracias!